Worst-case to Average-case Hardness of LWE:

A Simple and Practical Perspective

Alexandra Veliche

joint with Divesh Aggarwal & Leong Jin Ming

Roadmap

I. LWE Problem and Why it is Important

II. Alternative Perspective on Computational Hardness

III. Our Results

IV. Proof Techniques

Roadmap

I. LWE Problem and Why it is Important

II. Alternative Perspective on Computational Hardness

III. Our Results

IV. Proof Techniques

Learning With Errors

On Lattices, Learning with Errors, Random Linear Codes, and Cryptography

Oded Regev *

May 2, 2009

Abstract

Our main result is a reduction from worst-case lattice problems such as GAPSVP and SIVP to a certain learning problem. This learning problem is a natural extension of the 'learning from parity with error' problem to higher moduli. It can also be viewed as the problem of decoding from a random linear code. This, we believe, gives a strong indication that these problems are hard. Our reduction, however, is quantum. Hence, an efficient solution to the learning problem implies a *quantum* algorithm for GAPSVP and SIVP. A main open question is whether this reduction can be made classical (i.e., non-quantum).

We also present a (classical) public-key cryptosystem whose security is based on the hardness of the learning problem. By the main result, its security is also based on the worst-case quantum hardness of GAPSVP and SIVP. The new cryptosystem is much more efficient than previous lattice-based cryptosystems: the public key is of size $\tilde{O}(n^2)$ and encrypting a message increases its size by a factor of $\tilde{O}(n)$ (in previous cryptosystems these values are $\tilde{O}(n^4)$ and $\tilde{O}(n^2)$, respectively). In fact, under the assumption that all parties share a random bit string of length $\tilde{O}(n^2)$, the size of the public key can be reduced to $\tilde{O}(n)$.

Learning With Errors

[Regev, 2009] – reduction from worst-case lattice problems to decision-LWE

$$\boxed{\text{GapSVP}} \xrightarrow{\text{quantum}} \boxed{\text{BDD}} \xrightarrow{\text{classical}} \boxed{\text{LWE}}$$

Learning With Errors

[Peikert, 2009], [Brakerski+,2013] – fully classical reduction that preserves parameters

$$\boxed{\text{GapSVP}} \xrightarrow{\text{classical}} \boxed{\text{BDD}} \xrightarrow{\text{classical}} \boxed{\text{LWE}}$$

Learning With Errors

Worst-case hardness of standard lattice problems $\Longrightarrow$ Average-case hardness $\Longrightarrow$ Lattice-based cryptosystems

$\boxed{\text{GapSVP}}$ $\boxed{\text{BDD}}$ $\boxed{\text{LWE}}$

Learning With Errors

Worst-case hardness of standard lattice problems $\Longrightarrow$ Average-case hardness $\Longrightarrow$ Lattice-based cryptosystems

$\boxed{\text{GapSVP}}$ $\boxed{\text{BDD}}$ $\boxed{\text{LWE}}$

Post-quantum cryptography!

no known efficient quantum algorithms for these problems

Learning With Errors

$\mathbf{LWE}_{n,p,\phi}$: n = dimension, p = modulus, ϕ = error distribution over $\mathbb{R}/\mathbb{Z}$

Given n^k noisy samples of the form $\left(\mathbf{a}, b = \frac{1}{p}\langle \mathbf{a}, \mathbf{s}\rangle + e\right)$, where

$\mathbf{a} \leftarrow \mathbb{Z}_p^n$ uniformly random, $e \leftarrow \phi$ error, and $\mathbf{s} \in \mathbb{Z}_p^n$ unknown secret vector,

(if samples are uniformly random over $\mathbb{Z}_p^n \times \mathbb{R}/\mathbb{Z}$.

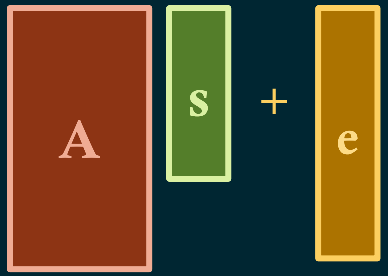
Learning With Errors

LWE$_{n,p,\phi}$: n = dimension, p = modulus, ϕ = error distribution over $\mathbb{R}/\mathbb{Z}$

Given n^k noisy samples of the form $\left(\mathbf{a}, b = \frac{1}{p}\langle \mathbf{a}, \mathbf{s} \rangle + e\right)$, where

$\mathbf{a} \leftarrow \mathbb{Z}_p^n$ uniformly random, $e \leftarrow \phi$ error, and $\mathbf{s} \in \mathbb{Z}_p^n$ unknown secret vector,

(if samples are uniformly random over $\mathbb{Z}_p^n \times \mathbb{R}/\mathbb{Z}$.



Learning With Errors

$\mathbf{LWE}_{n,p,\phi}$: n = dimension, p = modulus, ϕ = error distribution over $\mathbb{R}/\mathbb{Z}$

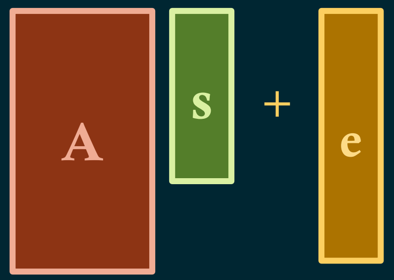
Given n^k noisy samples of the form $\left(\mathbf{a}, b = \frac{1}{p}\langle \mathbf{a}, \mathbf{s}\rangle + e\right)$, where

$\mathbf{a} \leftarrow \mathbb{Z}_p^n$ uniformly random, $e \leftarrow \phi$ error, and $\mathbf{s} \in \mathbb{Z}_p^n$ unknown secret vector,

(search-LWE) output $\mathbf{s}$.

(decision-LWE) output

- YES if samples are from the $\mathbf{LWE}$ distribution determined by $\mathbf{s}$ and ϕ,
- NO if samples are uniformly random over $\mathbb{Z}_p^n \times \mathbb{R}/\mathbb{Z}$.



Algorithms for LWE

[Blum-Kalai-Wasserman, 2000] – Best known algorithm for $\mathrm{LWE}_{n,p,\phi}$ runs in $p^{O(n/\log n)}$ time.

Algorithms for LWE

[Blum-Kalai-Wasserman, 2000] – Best known algorithm for $\mathrm{LWE}_{n,p,\phi}$ runs in $p^{O(n/\log n)}$ time.

$$\boxed{\mathrm{GapSVP}_\gamma} \xrightarrow{\text{classical}} \boxed{\mathrm{BDD}_\alpha} \xrightarrow{\text{classical}} \boxed{\mathrm{LWE}_{n^2,p}}$$

$\gamma = \mathrm{poly}(n)$

$p = \mathrm{poly}(n)$

Algorithms for LWE

[Blum-Kalai-Wasserman, 2000] – Best known algorithm for $\mathrm{LWE}_{n,p,\phi}$ runs in $p^{O(n/\log n)}$ time.

$$\boxed{\mathrm{GapSVP}_\gamma} \xrightarrow{\text{classical}} \boxed{\mathrm{BDD}_\alpha} \xrightarrow{\text{classical}} \boxed{\mathrm{LWE}_{n^2,p}}$$

$\gamma = \mathrm{poly}(n)$ $\qquad\qquad$ $p = \mathrm{poly}(n)$

If current known algorithms for these are the best possible $\Longrightarrow$ best known algorithm for this runs in $2^{\Omega(\sqrt{n})}$ time

Algorithms for LWE

[Blum-Kalai-Wasserman, 2000] – Best known algorithm for $\mathrm{LWE}_{n,p,\phi}$ runs in $p^{O(n/\log n)}$ time.

$$\boxed{\mathrm{GapSVP}_\gamma} \xrightarrow{\text{classical}} \boxed{\mathrm{BDD}_\alpha} \xrightarrow{\text{classical}} \boxed{\mathrm{LWE}_{n^2,p}}$$

$\gamma = \mathrm{poly}(n)$

$p = \mathrm{poly}(n)$

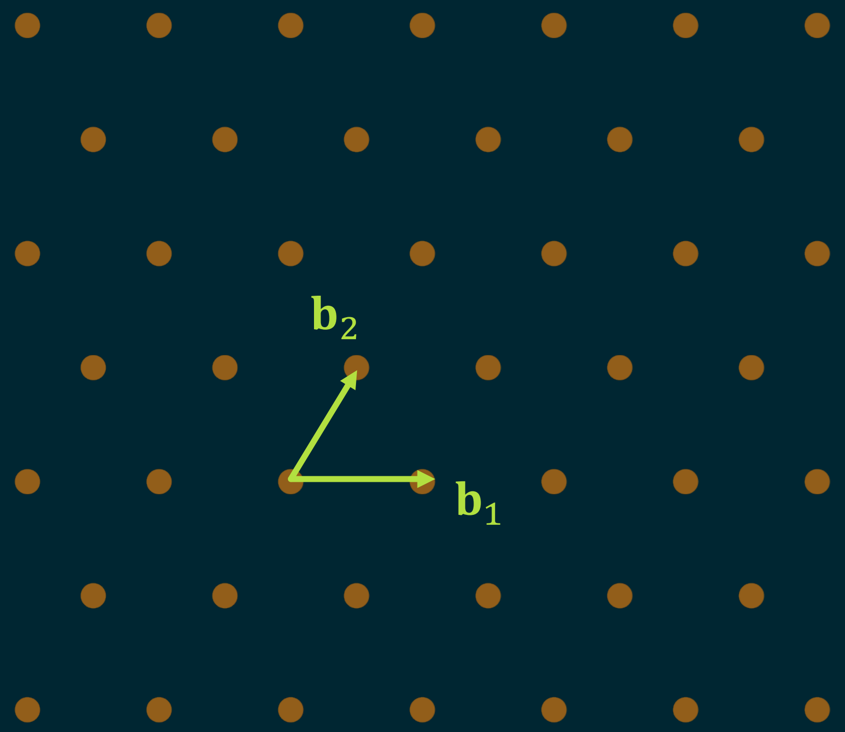
If current known algorithms for these are the best possible $\Longrightarrow$ best known algorithm for this runs in $2^{\Omega(\sqrt{n})}$ time

Gap!

Lattices

Lattice: A discrete additive subgroup of $\mathbb{R}^n$ consisting of all linear combinations of linearly independent basis vectors $\mathcal{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_\mathrm{n}\} \subset \mathbb{R}^n$:

$$\mathcal{L} = \mathcal{L}(\mathcal{B}) = \left\{ \sum_{i=1}^{n} a_i \mathbf{b}_i : a_1, \dots, a_n \in \mathbb{Z} \right\}.$$



Shortest Vector Problem

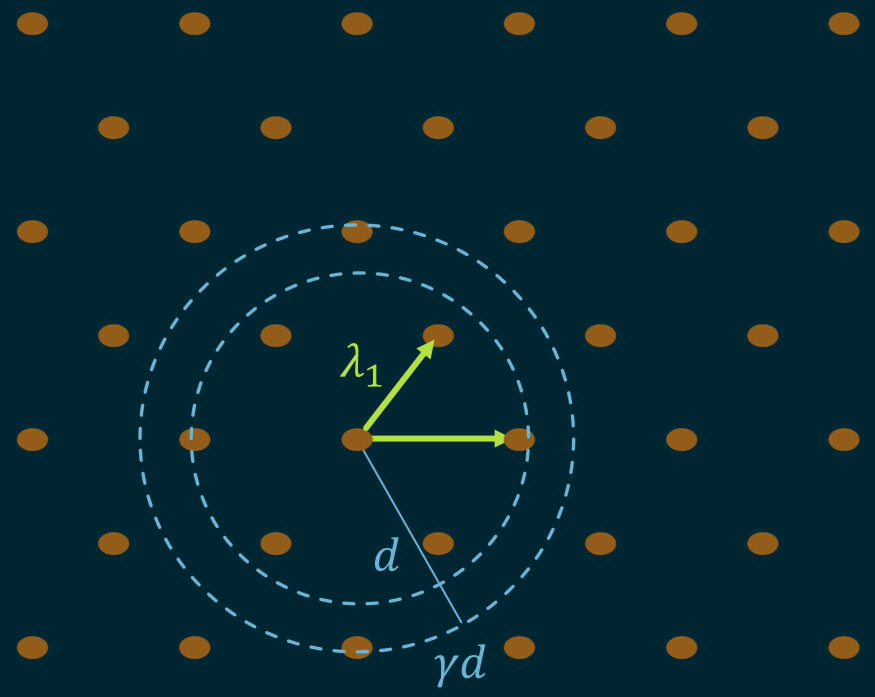
GapSVP$_\gamma$: $\gamma \geq 1$ approximation factor

Given a basis $\mathcal{B}$ for a full-rank lattice $\boldsymbol{\mathcal{L}} = \boldsymbol{\mathcal{L}}(\mathcal{B}) \subseteq \mathbb{R}^n$

and a distance parameter $d > 0$,

output

- YES if $\lambda_1(\boldsymbol{\mathcal{L}}) \leq d$
- NO if $\lambda_1(\boldsymbol{\mathcal{L}}) \geq \gamma \cdot d$.



Bounded Distance Decoding

$\mathbf{BDD}_\alpha$: $\alpha =$ approximation factor

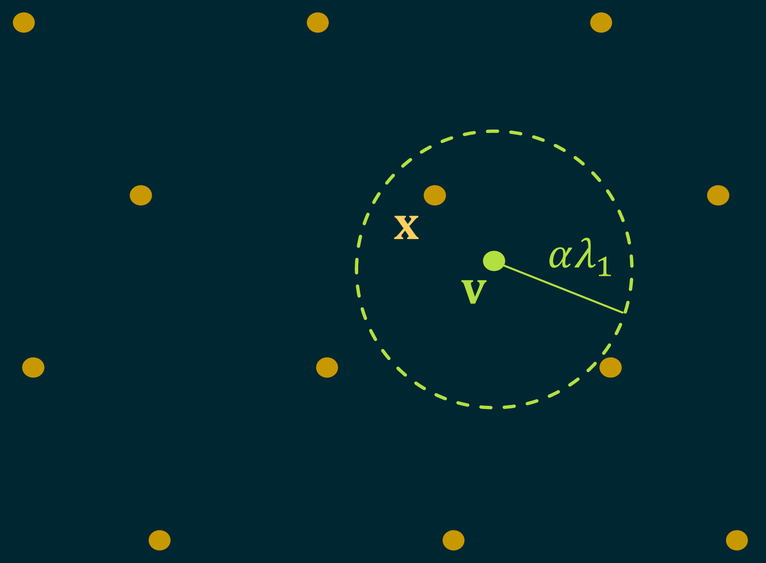
Given a basis $\mathcal{B}$ for a full-rank lattice $\boldsymbol{\mathcal{L}} = \boldsymbol{\mathcal{L}}(\mathcal{B}) \subseteq \mathbb{R}^n$

and a target vector $\mathbf{v} \in \mathbb{R}^n$,

with the promise that $dist(\mathbf{v}, \mathcal{L}) < \alpha \cdot \lambda_1(\mathcal{L})$,

find a lattice vector closest to $\mathbf{v}$,

i.e. $\mathbf{x} \in \mathcal{L}$ such that $\|\mathbf{v} - \mathbf{x}\| < \alpha \cdot \lambda_1(\mathcal{L})$.



Bounded Distance Decoding

BDD$_{\alpha}$: $\alpha \in \left(0, \frac{1}{2}\right)$ approximation factor

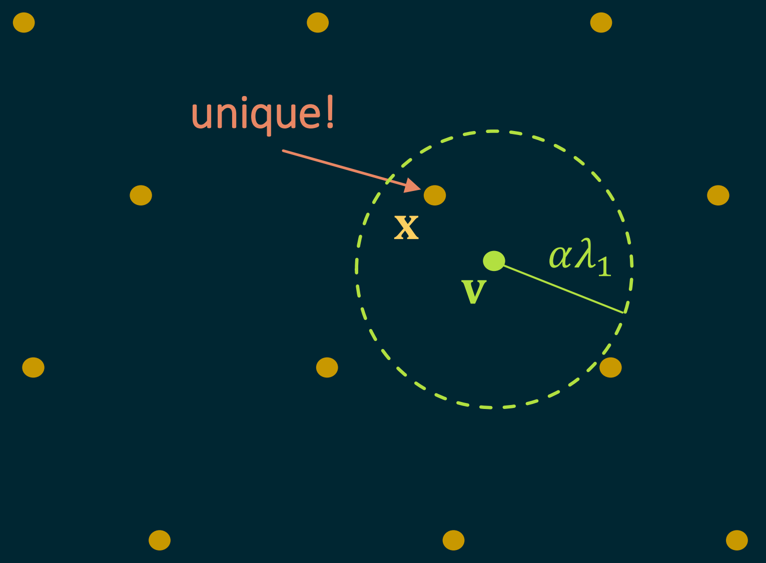
Given a basis $\mathcal{B}$ for a full-rank lattice $\boldsymbol{\mathcal{L}} = \boldsymbol{\mathcal{L}}(\mathcal{B}) \subseteq \mathbb{R}^n$

and a target vector $\mathbf{v} \in \mathbb{R}^n$,

with the promise that $dist(\mathbf{v}, \mathcal{L}) < \alpha \cdot \lambda_1(\mathcal{L})$,

find *the unique* lattice vector closest to $\mathbf{v}$,

i.e. $\mathbf{x} \in \mathcal{L}$ such that $\|\mathbf{v} - \mathbf{x}\| < \alpha \cdot \lambda_1(\mathcal{L})$.



Roadmap

I. LWE Problem and Why it is Important

II. Alternative Perspective on Computational Hardness

III. Our Results

IV. Proof Techniques

Bit Security

What does it mean for a cryptosystem to be 256-bit secure?

Bit Security

What does it mean for a cryptosystem to be 256-bit secure?

(a) The fastest algorithm for breaking the cryptosystem runs in 2^{256} time.

(b) An efficient algorithm succeeds in attacking the cryptosystem with probability $> 2^{-256}$.

Complexity Framework

[Aggarwal-Maurer, 2011] – *unpredictability entropy* of a computational problem $\mathcal{P}$:

If p is the maximum success probability of any PPT algorithm that solves $\mathcal{P}$,

then its unpredictability entropy is $\log_2 p$.

Complexity Framework

[Aggarwal-Maurer, 2011] – *unpredictability entropy* of a computational problem $\mathcal{P}$:

If p is the maximum success probability of any PPT algorithm that solves $\mathcal{P}$,

then its unpredictability entropy is $\log_2 p$.

Complexity Framework

Reduction algorithm for $\mathcal{P} \rightarrow \mathcal{Q}$ makes k calls to oracle for $\mathcal{Q}$.

Success probability of solving $\mathcal{P}$ is $\delta \implies$ success probability of solving $\mathcal{Q}$ is $\delta^{1/k}$.

Complexity Framework

Reduction algorithm for $\mathcal{P} \to \mathcal{Q}$ makes k calls to oracle for $\mathcal{Q}$.

Success probability of solving $\mathcal{P}$ is $\delta \implies$ success probability of solving $\mathcal{Q}$ is $\delta^{1/k}$.

Use only one-shot reductions!

Reduction from Lattice Problems

Success probability of any PPT algorithm for $\mathrm{LWE}_{n,p}$ is p^{-n}.

$$\boxed{\mathrm{GapSVP}_\gamma} \xrightarrow{\text{classical}} \boxed{\mathrm{BDD}_\alpha} \xrightarrow{\text{classical}} \boxed{\mathrm{LWE}_{n^2,p,\phi}}$$

$\gamma = \mathrm{poly}(n)$

$p = \mathrm{poly}(n)$

Reduction from Lattice Problems

Success probability of any PPT algorithm for $\mathrm{LWE}_{n,p}$ is p^{-n}.

$$\boxed{\mathrm{GapSVP}_\gamma} \xrightarrow{\text{classical}} \boxed{\mathrm{BDD}_\alpha} \xrightarrow{\text{classical}} \boxed{\mathrm{LWE}_{n^2,p,\phi}}$$

$\gamma = \mathrm{poly}(n)$

$p = \mathrm{poly}(n)$

Success probability of best known PPT algorithm for $\mathrm{GapSVP}_{n,\gamma}$ is $2^{-\Theta(n^2/\log n)}$.

Complexity Framework

Conjecture:

For any constants $c, c' > 0$, there is exists $\kappa = \kappa(c, c') > 0$ such that

no algorithm can solve BDD_α on an arbitrary n-rank lattice for $\alpha = \frac{1}{n^c}$ in time $n^{c'}$

with success probability better than $2^{-n^2/\kappa\ \log n}$.

Complexity Framework

Conjecture:

For any constants $c, c' > 0$, there is exists $\kappa = \kappa(c, c') > 0$ such that

no algorithm can solve BDD_α on an arbitrary n-rank lattice for $\alpha = \frac{1}{n^c}$ in time $n^{c'}$

with success probability better than $2^{-n^2/\kappa \ \log n}$.

Assuming this conjecture, can we show a lower bound of $p^{-\Omega(n)}$

on the success probability of any PPT algorithm that solves $\mathrm{LWE}_{n,p}$?

Roadmap

I. LWE Problem and Why it is Important

II. Alternative Perspective on Computational Hardness

III. Our Results

IV. Proof Techniques

Our Results

Theorem 1: *(informal)* If no PPT algorithm can solve BDD_α for approximation factor $\alpha \in \left(0, \frac{1}{2}\right)$

with success probability greater than $2^{-\frac{n^2}{\kappa \log n} - 4}$ for some $\kappa > 0$,

then no PPT algorithm can solve search-$\mathrm{LWE}_{n,p,\phi}$ with binary secret, dimension n, and

modulus $p = \mathrm{poly}(n)$ with success probability $2^{-\frac{n^2}{\kappa \log n}}$.

Main Problem

[Aggarwal-Maurer, 2011] – *unpredictability entropy* of a computational problem $\mathcal{P}$:

If p is the maximum success probability of any PPT algorithm that solves $\mathcal{P}$,

then its unpredictability entropy is $\log_2 p$.

What does this mean for decision problems?

Learning With Errors

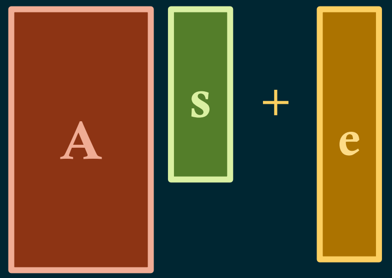
decision-LWE$_{n,p,\phi}$: n = dimension, p = modulus, ϕ = error distribution over $\mathbb{R}/\mathbb{Z}$

Given n^k noisy samples of the form $\left(\mathbf{a}, \frac{1}{p}\langle \mathbf{a}, \mathbf{s} \rangle + e\right)$, where $\mathbf{a} \leftarrow \mathbb{Z}_p^n$ uniformly random,

$e \leftarrow \phi$ error, and $\mathbf{s} \in \mathbb{Z}_p^n$ unknown secret vector,

output

- YES if samples are from the LWE distribution determined by $\mathbf{s}$ and ϕ,
- NO if samples are *uniformly random* over $\mathbb{Z}_p^n \times \mathbb{R}/\mathbb{Z}$.



Complexity Framework

If there is no PPT algorithm that solves search-$\mathrm{LWE}_{n,p}$ with success probability α,

then can we show that there is no algorithm that solves decision-$\mathrm{LWE}_{n,p}$ and outputs

- YES with probability $\alpha' \approx \alpha$ given a YES instance
- YES with probability $\beta' \ll \alpha'$ given a NO instance?

Complexity Framework

If there is no PPT algorithm that solves search-$\mathrm{LWE}_{n,p}$ with success probability α,

then can we show that there is no algorithm that solves decision-$\mathrm{LWE}_{n,p}$ and answers

- Correctly with probability $\approx \alpha$
- Incorrectly with probability $\beta \approx 0$
- $\perp$ with probability $\approx 1 - \alpha$?

Our Results

Theorem 2: *(informal)* If no algorithm can solve search-$\mathrm{LWE}_{n,p}$ for modulus $p = \mathrm{poly}(n)$

with success probability α in *expected* polynomial time,

then no PPT algorithm can solve decision-$\mathrm{LWE}_{n,p}$ for modulus p that

- outputs the correct answer with probability α and
- outputs $\perp$ with probability $1 - \alpha$.

Roadmap

I. LWE Problem and Why it is Important

II. Alternative Perspective on Computational Hardness

III. Our Results

IV. Proof Techniques

Chain of Reductions

I.

$\boxed{\mathrm{BDD}_{\alpha}} \longrightarrow \boxed{\text{mod-BDD}_{\alpha,p}} \longrightarrow \boxed{\text{gen-LWE}_{n,p,\mathcal{D}}} \longrightarrow \boxed{\mathrm{LWE}_{n^2,p}}$

Modular Bounded Distance Decoding

mod-BDD$_{\alpha,p}$: α = approximation factor, p = modulus

Given a basis $\mathcal{B}$ for a full-rank lattice $\boldsymbol{\mathcal{L}} = \boldsymbol{\mathcal{L}}(\mathcal{B}) \subseteq \mathbb{R}^n$

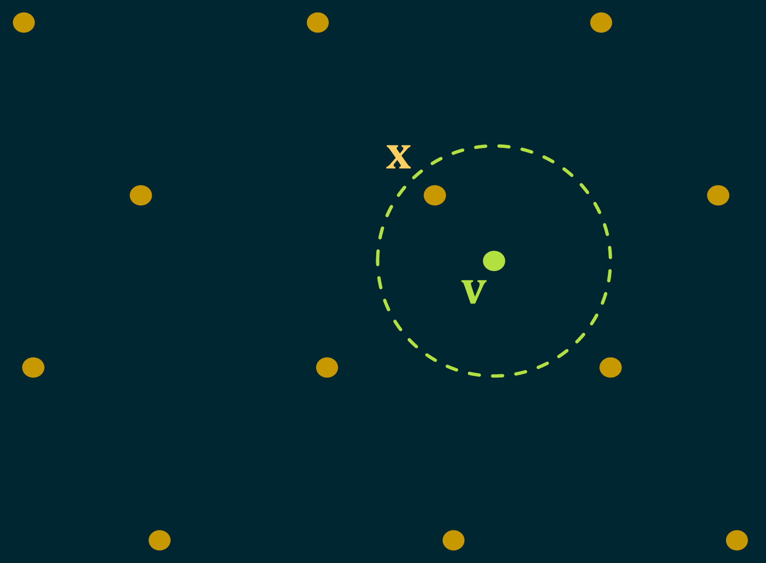
and a target vector $\mathbf{v} \in \mathbb{R}^n$,

with the promise that $dist(\mathbf{v}, \mathcal{L}) < \alpha \cdot \lambda_1(\mathcal{L})$,

find the coefficient vector of a lattice vector closest to $\mathbf{v} \bmod p$,

i.e. $\mathbf{B}^{-1}\mathbf{x} \bmod p \in \mathbb{Z}_p^n$ for some

$\mathbf{x} \in \mathcal{L}$ such that $\|\mathbf{v} - \mathbf{x}\| < \alpha \cdot \lambda_1(\mathcal{L})$.



Chain of Reductions

I.

$$\boxed{\mathrm{BDD}_\alpha} \longrightarrow \boxed{\text{mod-}\mathrm{BDD}_{\alpha,p}} \longrightarrow \boxed{\text{gen-}\mathrm{LWE}_{n,p,\mathcal{D}}} \longrightarrow \boxed{\mathrm{LWE}_{n,p,\phi}}$$

Blow up the modulus!
use $p = 2^n$

Chain of Reductions

I.

$\boxed{\mathrm{BDD}_{\alpha}} \longrightarrow \boxed{\text{mod-}\mathrm{BDD}_{\alpha,p}} \longrightarrow \boxed{\text{gen-}\mathrm{LWE}_{n,p,\mathcal{D}}} \longrightarrow \boxed{\mathrm{LWE}_{n,p,\phi}}$

Blow up the modulus!
use $p = 2^n$

success prob. q $\longleftarrow$ success prob. q

Chain of Reductions

I.

$$\boxed{\mathrm{BDD}_{\alpha}} \longrightarrow \boxed{\text{mod-BDD}_{\alpha,p}} \longrightarrow \boxed{\text{gen-LWE}_{n,p,\mathcal{D}}} \longrightarrow \boxed{\mathrm{LWE}_{n,p,\phi}}$$

Generate discrete Gaussian
samples and use them to
generate LWE samples

Chain of Reductions

I.

$\mathrm{BDD}_{\alpha} \longrightarrow \text{mod-}\mathrm{BDD}_{\alpha,p} \longrightarrow \text{gen-}\mathrm{LWE}_{n,p,\mathcal{D}} \longrightarrow \mathrm{LWE}_{n,p,\phi}$

Generate discrete Gaussian samples and use them to generate LWE samples

success prob. $q - \epsilon$ $\longleftarrow$ success prob. q

Chain of Reductions

I.

$$\boxed{\mathrm{BDD}_{\alpha}} \longrightarrow \boxed{\text{mod-}\mathrm{BDD}_{\alpha,p}} \longrightarrow \boxed{\text{gen-}\mathrm{LWE}_{n,p,\mathcal{D}}} \longrightarrow \boxed{\mathrm{LWE}_{n,p,\phi}}$$

Carefully sample Gaussian noise that guarantees optimal success probability

Chain of Reductions

I.

$$\boxed{\mathrm{BDD}_\alpha} \longrightarrow \boxed{\text{mod-}\mathrm{BDD}_{\alpha,p}} \longrightarrow \boxed{\text{gen-}\mathrm{LWE}_{n,p,\mathcal{D}}} \longrightarrow \boxed{\mathrm{LWE}_{n,p,\phi}}$$

Carefully sample Gaussian noise that guarantees optimal success probability

success prob. $\frac{q}{(1+\epsilon)^3}$ $\longleftarrow$ success prob. q

Chain of Reductions

I.

$$\begin{tikzcd}
\boxed{\mathrm{BDD}_{\alpha}} \arrow[r] & \begin{array}{c}\boxed{\text{mod-BDD}_{\alpha,p}}\\ \text{modulus}\\ p = 2^n\end{array} \arrow[r] & \begin{array}{c}\boxed{\text{gen-LWE}_{n,p,\mathcal{D}}}\\ \text{modulus}\\ p = 2^n\end{array} \arrow[r] & \begin{array}{c}\boxed{\mathrm{LWE}_{n,p,\phi}}\\ \text{modulus}\\ p = 2^n\end{array} \arrow[d] \\
 & & \begin{array}{c}\boxed{\mathrm{LWE}_{n^2,p,\phi}}\\ \text{modulus}\\ p = n^{\Theta(1)}\end{array} & \begin{array}{c}\boxed{\text{binary-LWE}_{n^2,p,\phi}}\\ \text{modulus}\\ p = n^{\Theta(1)}\end{array} \arrow[l]
\end{tikzcd}$$

Chain of Reductions

I.

$$\begin{tikzcd}
\boxed{\mathrm{BDD}_{\alpha}} \arrow[r] & \boxed{\text{mod-BDD}_{\alpha,p}} \arrow[r] & \boxed{\text{gen-LWE}_{n,p,\mathcal{D}}} \arrow[r] & \boxed{\mathrm{LWE}_{n,p,\phi}} \arrow[d] \\
 & & \boxed{\mathrm{LWE}_{n^2,p,\phi}} & \boxed{\text{binary-LWE}_{n^2,p,\phi}} \arrow[l]
\end{tikzcd}$$

$\text{mod-BDD}_{\alpha,p}$: modulus $p = 2^n$

$\text{gen-LWE}_{n,p,\mathcal{D}}$: modulus $p = 2^n$

$\mathrm{LWE}_{n,p,\phi}$: modulus $p = 2^n$, success prob.q

$\text{binary-LWE}_{n^2,p,\phi}$: modulus $p = n^{\Theta(1)}$, success prob.q

$\mathrm{LWE}_{n^2,p,\phi}$: modulus $p = n^{\Theta(1)}$, success prob.q

Chain of Reductions

II.

$\boxed{\text{search-LWE}_{n^2,p,\phi}} \longrightarrow \boxed{\text{GL-LWE}_{n,p,\phi}} \longrightarrow \boxed{\text{decision-LWE}_{n,p,\phi}}$

Goldreich-Levin LWE

GL-LWE$_{n,p,\phi}$: n = dimension, p = modulus, ϕ = error distribution over $\mathbb{R}/\mathbb{Z}$

Given n^k noisy samples of the form $\left(\mathbf{a}, \frac{1}{p}\langle\mathbf{a},\mathbf{s}\rangle + e\right)$,

where $\mathbf{a} \leftarrow \mathbb{Z}_p^n$ uniformly random, $e \leftarrow \phi$ error, and $\mathbf{s} \in \mathbb{Z}_p^n$ fixed secret vector,

and a $\mathbf{r} \in \mathbb{Z}_p^n$ uniformly random vector,

output $\langle\mathbf{s},\mathbf{r}\rangle \bmod p$.

Goldreich-Levin Theorem

Goldreich-Levin Theorem: *(informal)*

Given any one-way function f, the function h defined by $h(\mathrm{x}, \mathrm{r}) = \langle \mathrm{x}, \mathrm{r} \rangle \bmod 2$

is a hard-core predicate for the function g defined by $g(\mathrm{x}, \mathrm{r}) = (f(\mathrm{x}), \mathrm{r})$.

Goldreich-Levin Theorem

Levin's Theorem: *(informal)*

Given any one-way function f, the function h defined by $h(\mathrm{x},\mathrm{r}) = \langle \mathrm{x},\mathrm{r}\rangle \bmod 2$

is a hard-core predicate for the function g defined by $g(\mathrm{x},\mathrm{r}) = (f(\mathrm{x}),\mathrm{r})$.

The maximum success probability of finding $h(\mathrm{x},\mathrm{r})$ given $f(\mathrm{x}),\mathrm{r}$ is

the maximum success probability of finding x given $f(\mathrm{x})$.

Goldreich-Levin Theorem

Theorem: *(generalization)*

Let $p > 10.$ Given any one-way function $f,$ the function h defined by $h(\mathrm{x}, \mathrm{r}) = \langle \mathrm{x}, \mathrm{r} \rangle \bmod p$ is a hard-core predicate for the function g defined by $g(\mathrm{x}, \mathrm{r}) = (f(\mathrm{x}), \mathrm{r})$.

The maximum success probability of finding $h(\mathrm{x}, \mathrm{r})$ given $f(\mathrm{x}), \mathrm{r}$ is the maximum success probability of finding x given $f(\mathrm{x})$.

Chain of Reductions

II.

$$\text{search-LWE}_{n^2,p,\phi} \longrightarrow \text{GL-LWE}_{n,p,\phi} \longrightarrow \text{decision-LWE}_{n,p,\phi}$$

Use the GL reduction!

Chain of Reductions

II.

search-LWE$_{n^2,p}$ $\longrightarrow$ GL-LWE$_{n,p}$ $\longrightarrow$ decision-LWE$_{n,p}$

Re-randomize samples
and run oracle on them

Future Directions

- The two chains of reductions are disconnected, because *expected* polynomial-time reduction is a fundamental part of the GL theorem. Is there a workaround?
- Establish a similar result for GapSVP to complete the chain of reductions to BDD.
- Use this alternative framework to study the complexity of other computational problems relevant to cryptography or learning problems.

Future Directions

- The two chains of reductions are disconnected, because *expected* polynomial-time reduction is a fundamental part of the GL theorem. Is there a workaround?
- Establish a similar result for GapSVP to complete the chain of reductions to BDD.
- Use this alternative framework to study the complexity of other computational problems relevant to cryptography or learning problems.

Thank you! Questions?